



Below is the **institutional redesign of the entry engine** for the CondorNet™ Automated Trading System Options Research Platform (CATSOR), aligned with our **v4.5 backtester architecture** and the requirements:

- multiple simultaneous positions
- portfolio-aware allocation
- strategy enumeration across 58 Options strategies (and Custom CNv45 strategy creation)
- real-time candidate ranking and selection by process of elimination
- capital constraints and institutional risk management controls
- dataset generation for policy learning

This replaces the **single-entry gating logic** and becomes the **portfolio entry allocator**. The outputs will be stored to file for each strategy separately, then like the reversal pivots identified, the dataset will be appended with the strategies selected and synchronized with the m1, m5, m15, h1, and options chain data sets by date and time of entry and exit. The model generation process will then be enhanced substantially as it will learn what it did right and what it did wrong in terms of its decision processing, and take corrective measures, re-compute its decision tree logic, and output a new model file. The new model file will then be tested. This loop will continue until it does not make a mistake on any SPY options and SPOT data sets its ingesting. Currently, we have 20 years of historical 1 minute bar SPY Options and SPOT data sets, we have all the precompute scripts to populate the fields, and we have the scripts written in Trading View to output the IVR based and ranked pivot reversal signals which do not repaint, and are working to perfection – it has been manually tested and thus far in the CME Professional Futures Trading Competition, this custom pine script indicator I first created in 2018 has been fully updated and can be applied on any instrument at any timeframe, and predict the reversals so well, its *almost unbelievable*. This has given me a competitive advantage when manually trading in the CME competition and thus far out of > 500 positions taken, 5 have been losers from hitting the stop loss which I set super tight when I trade as I never allow for capital to be placed at risk of a gapper out of the blue to happen in the wrong direction, and hence why my trading logs will show very high net profits, with very low loss. In the past 2 weeks of testing the new indicator, the max draw down has been \$750.00 on a \$100K account leveraged 2:1. The max NP has been \$70,000.00 an outlier on the Nikkei Japanese Index. The average winning position is \$2500.00.

Note: People are wanting to use the “bot” yet we cannot even afford to get the Trading View monthly subscription for my account to output the data needed for the data sets to retrain the model file for the next set of sequential tests. That is illogical to think something can be completed for use, without having what is required to complete the mission. It’s like a group of people sitting around the table with the know-how to put a man on the moon, have the detailed blueprint to do so, yet not enough money to build the rocket. If people want to use this new advanced system for profits, and “let’s see how it performs”, let them know, these are fixed costs and ongoing expenses so they can do that, give them your baking info, and have them loan it for first right to use the software, period. As the money they will make from the software being completed - multiples greater than the *pocket change* by comparison to completing the mission. We are 95% of the way there, *not just getting started*. If people try to rake our balls across the ground, and ask for too much for so little, **tell them to kick rocks**, as we will get this done, maybe a bit slower that’s all. Then they can kick back and wish they jumped on board with us.

ENTRY ENGINE ARCHITECTURE

(Candidate Pool → Ranking → Capital Allocation)

1. ENTRY ENGINE OBJECTIVE

At every simulation timestep t , the engine must determine:

$$\mathcal{A}_t = \{a_1, a_2, \dots, a_k\}$$

where

$$a_i \in \{ENTER, HOLD, EXIT, MODIFY, ADD, ABSTAIN\}$$

for **each strategy candidate** under the portfolio constraints.

The system therefore becomes a **multi-strategy portfolio control system**, not a single trade selector.

2. ACCOUNT AND CAPITAL MODEL

2.1 Initial Equity

$$E_0 = 100000$$

2.2 Buying Power

Options level-3 margin with 2:1 leverage

$$\begin{aligned} BP &= 2E_0 \\ BP &= 200000 \end{aligned}$$

2.3 Maximum Deployable Margin

Only **40%** of buying power may be deployed.

$$\begin{aligned}BP_{deploy}^{max} &= 0.40 \times BP \\ BP_{deploy}^{max} &= 80000\end{aligned}$$

2.4 Remaining Capital

Let

$$M_t = \sum_{i=1}^{N_t} margin_i$$

Then remaining deployable capital is

$$BP_{remaining}(t) = BP_{deploy}^{max} - M_t$$

3. STRATEGY ENUMERATION

At each timestep the engine enumerates

$$S = \{s_1, s_2, \dots, s_{50+}\}$$

Each strategy template contains

```
template_id  
legs  
margin_type  
max_positions  
cooldownBars  
entry_threshold  
pop_threshold  
stop_loss_mult  
profit_target  
margin_pct
```

4. PARAMETER GRID EXPANSION

Each strategy produces a parameter set

$$\theta = (DTE, \Delta, Width, ExitRule)$$

Example Iron Condor grid

$$\begin{aligned} DTE &\in \{7,14,21,30\} \\ \Delta &\in \{0.10,0.16,0.20\} \\ Width &\in \{5,10,15\} \\ Exit &\in \{50\%, 21DTE, \Delta breach\} \end{aligned}$$

Total combinations

$$|\Theta| = 108$$

5. CANDIDATE GENERATION

For each strategy s_i and parameter set θ_j :

Construct a **candidate trade**

$$c_{i,j}$$

Candidate contains

```
strategy_id  
parameters  
leg_symbols  
margin_required  
credit  
pop  
delta  
gamma  
theta  
vega
```

Total candidate set

$$\mathcal{C}_t = \{c_1, c_2, \dots, c_n\}$$

6. STAGE-1 FILTER

MODEL ELIGIBILITY

The CondorNet policy outputs

$$\pi_t = (a_1, a_2, \dots, a_k)$$

Relevant gates:

Entry logit

$$g_{entry} = \sigma(l_{entry})$$

Entry allowed if

$$g_{entry} > \tau_e$$

Probability of Profit Gate

$$POP = \sigma(l_{pop})$$

Candidate passes if

$$POP > \tau_{pop}$$

Default

$$\tau_{pop} = 0.75$$

Candidates passing become

$$C_1$$

7. STAGE-2 FILTER

COLLATERAL CONSTRAINT

Each candidate has margin

$$M_i$$

If

$$M_i > BP_{remaining}$$

candidate removed.

Remaining pool

$$\mathcal{C}_2$$

8. STAGE-3 FILTER

EXECUTION FEASIBILITY

Multi-leg order must satisfy

Spread constraint

$$spread\% < \tau_{spread}$$

Where

$$spread\% = \frac{ask - bid}{mid}$$

Mid

$$mid = \frac{ask + bid}{2}$$

Atomicity constraint

All legs must fill.

If not

candidate rejected.

Remaining set

$$\mathcal{C}_3$$

9. STAGE-4 RANKING

Each candidate receives a ranking score.

Define

Expected value

$$EV_i = POP_i \cdot Credit_i - (1 - POP_i) \cdot Loss_i$$

Risk adjusted utility

$$U_i = \frac{EV_i}{Margin_i}$$

But we also incorporate a **slider parameter**

$$\lambda \in [0,1]$$

where

$$\lambda = 0$$

means maximize probability of success

$$\lambda = 1$$

means maximize return.

Score function

$$Score_i = \lambda \cdot EV_i + (1 - \lambda) \cdot POP_i - \alpha \cdot Spread_i - \beta \cdot TailRisk_i$$

10. CANDIDATE SORTING

Sort candidates by

$$Score_i$$

descending.

11. PORTFOLIO ALLOCATION

We allocate candidates sequentially while margin remains.

Pseudo-algorithm

```
portfolio = []  
for candidate in sorted_candidates:  
    if candidate.margin <= BP_remaining:  
        open_trade(candidate)  
        BP_remaining -= candidate.margin
```

12. MARGIN CALCULATION

Defined-risk spread margin

$$Margin = (Width - Credit) \times 100 \times Contracts$$

Example

Width

$$W = 10$$

Credit

$$C = 2$$

Contracts

$$Q = 5$$

Margin

$$\begin{aligned} \text{Margin} &= (10 - 2) \times 100 \times 5 \\ \text{Margin} &= 4000 \end{aligned}$$

13. POSITION SIZE SOLVER

For remaining margin

$$BP_{remaining}$$

max contracts

$$Q_{max} = \left\lfloor \frac{BP_{remaining}}{\text{Margin}_{per\ c} \times \text{contract}} \right\rfloor$$

14. OPEN TRADE EVENT

Once allocated

create trade record

```
Trade(  
trade_id  
strategy  
entry_dt  
legs  
credit  
margin  
dte  
greeks  
policy_scores  
)
```

Also emit event

```
{  
  action:"OPEN",  
  trade_id,  
  timestamp,  
  parameters,  
  policy_scores  
}
```

This becomes part of the **trajectory dataset**.

15. ENTRY TRACE LOGGING (CRITICAL FOR TRAINING)

Every rejected candidate logs reason code

```
ENTRY_REJECT_POLICY  
ENTRY_REJECT_POP  
ENTRY_REJECT_MARGIN  
ENTRY_REJECT_LIQUIDITY  
ENTRY_REJECT_ATOMICITY  
ENTRY_REJECT_SPREAD
```

These logs allow the model to later learn:

- why trades were skipped
- why trades were selected

16. FINAL ENTRY ENGINE LOOP

Pseudo-structure

```
candidates = enumerate_candidates()  
stage1 = filter_policy(candidates)  
stage2 = filter_margin(stage1)  
stage3 = filter_execution(stage2)  
ranked = score_candidates(stage3)  
allocate_portfolio(ranked)
```

17. DATASET RECORDING

For every timestep

record

$$(s_t, a_t, r_t, s_{t+1})$$

Where

State

$$s_t = (\text{spot}, \text{regime}, \text{greeks}, \text{position } s_t, \text{tate})$$

Action

$$a_t \in \{ENTER, HOLD, EXIT, MODIFY, ADD, ABSTAIN\}$$

Reward

$$r_t = \Delta PnL$$

18. WHY THIS DESIGN IS REQUIRED

The old logic

```
if len(open_trades)==0:
```

produces a **single-trade system**.

But the correct system must behave like a **portfolio optimizer**.

The new system performs

1. strategy discovery
2. candidate filtering
3. risk-aware ranking
4. capital allocation

which matches **institutional volatility desks**.

NEXT STEP

Now that the **entry engine is defined**, the next critical piece is:

The Trajectory Dataset Schema.

That determines **what exactly the model learns from the simulation.**

Without the correct schema the model will:

- memorize trades
- fail to learn hold/modify behavior
- fail to learn abstention.

Next I will produce the **complete CondorNet v4.5 trajectory dataset schema (institutional RL format)** that captures:

- entry logic traces
- strategy parameters
- greeks surfaces
- portfolio context
- execution realism
- rewards.

That dataset is the **single most important artifact in the entire platform.**

That is why when I first told everyone that data is king and is by far the most important part of everything, because what we feed the neural net creation process is the most critical factor, especially in how we feed the neural net. It's important that we datetime align everything, especially if we're using multiple datasets which 1 anchor is required. The chosen time frame for this is the M5 time frame. As such I will make sure with several pytests that everything is 100% completely In Sync and that the data is being ingested correctly and that the model is interpreting the data correctly and in doing so several outputs are needed during model training and need to watch it in real time you can't just let the trainer run, walk away and come back – At this level, every single detail of I/O is mission critical, just one singular field of a 10M row 90 column CSV file, that is not updated correctly, or somehow slipped by the tests, can throw the entire model creation process off - this takes a lot of time, a lot of effort, and 100% laser light focus with extreme attention to details and computational outputs from the GPU CUDA kernels.